

ChatGPT/LLM Log - Document all prompts and output

Lawrence Winters (lawrence.winters@digipen.edu)

Jacob Schertz (jacob.schertz@digipen.edu)

Durga Sandeep (d.namballa@digipen.edu)

Prompts:

ChatGPT ▾

how do I extract audio features in python?

Extracting audio features in Python is commonly done using libraries like **Librosa**, **PyDub**, **scipy**, or **torchaudio** (for deep learning). Here's a quick guide using **Librosa**, one of the most popular libraries:

🔧 Step 1: Install Required Libraries

```
bash
pip install librosa numpy matplotlib
```

🔊 Step 2: Load and Analyze an Audio File

```
python
...

```

ChatGPT ▾

🔊 Step 2: Load and Analyze an Audio File

```
python
import librosa
import librosa.display
import numpy as np
import matplotlib.pyplot as plt

# Load audio file
audio_path = 'your_audio_file.wav'
y, sr = librosa.load(audio_path, sr=None) # y: audio time series, sr: sampling rate
```

🔍 Step 3: Extract Common Audio Features

1. MFCC (Mel-Frequency Cepstral Coefficients)

```
python
mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
print(mfccs.shape) # (n_mfcc, n_frames)
```

ChatGPT

2. Spectral Centroid

```
python

spectral_centroid = librosa.feature.spectral_centroid(y=y, sr=sr)
```

Copy Edit

Log in

Sign up for free



3. Chroma Features

```
python

chroma_stft = librosa.feature.chroma_stft(y=y, sr=sr)
```

Copy Edit

4. Zero-Crossing Rate

```
python

zcr = librosa.feature.zero_crossing_rate(y)
```

Copy Edit

5. Spectral Rolloff

```
python

rolloff = librosa.feature.spectral_rolloff(y=y, sr=sr)
```

Copy Edit



ChatGPT

6. Tempo (BPM)

```
python

tempo, _ = librosa.beat.beat_track(y=y, sr=sr)
```

Copy Edit

Log in

Sign up for free



Step 4: Optional - Visualize the Features

```
python

plt.figure(figsize=(10, 4))
librosa.display.specshow(mfccs, x_axis='time')
plt.colorbar()
plt.title('MFCC')
plt.tight_layout()
plt.show()
```

Copy Edit

Optional: Feature Aggregation



Optional: Feature Aggregation

If you're using these features for machine learning, aggregate them (e.g., mean, std) to create fixed-length feature vectors:

```
python

mfcc_mean = np.mean(mfccs, axis=1)
mfcc_std = np.std(mfccs, axis=1)
```

Copy Edit

Log in

Sign up for free



Other Tools

- **PyDub** – for audio manipulation.
- **torchaudio** – if working with PyTorch.
- **scipy.io.wavfile** – basic feature extraction.

Would you like a full script or feature extraction tailored for speech, music, or ML?

Copy Edit

ChatGPT was used for learning how to extract features from audio files. This was completely new territory for me, so I felt that using ChatGPT to give myself a push in the right direction was highly valuable.

Lawrence.Winters:

ChatGPT ▾ Share ...

How can I make a horizontal bar graph for feature importance when using sklearn's RandomForestClassifier

To make a horizontal bar graph for feature importances from `sklearn`'s `RandomForestClassifier`, you can use `matplotlib` (or `seaborn`, if you prefer). Here's how you can do it using `matplotlib`:

✓ Code Example:

python Copy Edit

```
import matplotlib.pyplot as plt
import numpy as np
```

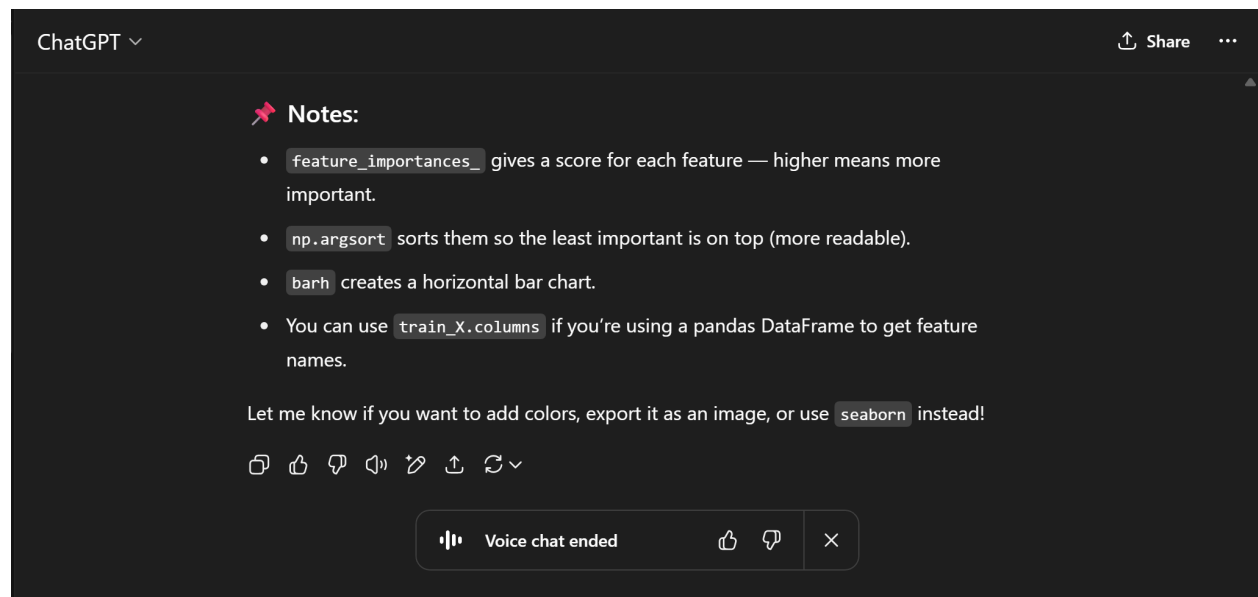
ChatGPT ▾ Share ...

Assuming `random\_forest` is your trained model and `train\_X` is Copy Edit

```
importances = random_forest.feature_importances_
feature_names = train_X.columns

# Sort features by importance
indices = np.argsort(importances)

# Plot horizontal bar chart
plt.figure(figsize=(10, 6))
plt.barh(range(len(importances)), importances[indices], align='center')
plt.yticks(range(len(importances)), [feature_names[i] for i in indices])
plt.xlabel('Feature Importance')
plt.title('Random Forest Feature Importances')
plt.tight_layout()
plt.show()
```



Explanation:

I used ChatGPT for this section of code because I didn't know how to get the feature importance values from the random forest classifier. Once I had that, I knew how to make a bar graph using matplotlib but did not know how to make a horizontal bar graph so I included that in my prompt.

I used one-hot encoding through `pd.dummies`, is there a way to get the labels back to a usable form for classification report since it is giving me errors using the current `train_y`

Yes — when using one-hot encoding (via `pd.get_dummies`) for your labels, you **cannot** pass the one-hot encoded labels directly into `classification_report()`, because it expects **string or integer class labels**, not binary vectors.

Here's how to fix it:

✅ Step-by-Step Fix

You already created one-hot encoded `train_y` and `test_y` like this:

```
python                                     Copy Edit
encoded_y = pd.get_dummies(df_full["Emotion"], prefix='Emotion', dtype=int)
```

Then you split:

```
python                                     Copy Edit
train_y, test_y = ...
```

Now, convert your one-hot labels back into **string** labels using `idxmax()` and `str.replace()`:

```
python                                     Copy Edit
# Get string labels from one-hot encoded columns
train_y_labels = train_y.idxmax(axis=1).str.replace("Emotion_", "")
test_y_labels = test_y.idxmax(axis=1).str.replace("Emotion_", "")
```

Then, after making predictions:

```
python                                     Copy Edit
pred_y = random_forest.predict(test_X)
```

Convert `pred_y` back to labels as well if needed (depending on whether you trained the model on label-encoded or one-hot targets).

Finally, pass string labels to the classification report:

ChatGPT

Share

python Copy Edit

```
from sklearn.metrics import classification_report

print(classification_report(test_y_labels, pred_y))
```

Why This Works

- `train_y` is a DataFrame with columns like `Emotion_Happy`, `Emotion_Angry`, etc.
- `.idxmax(axis=1)` gives the column name with the 1 (i.e., the correct class) for each row.
- `.str.replace("Emotion_", "")` strips the `"Emotion_"` prefix so you get back the actual label like `"Happy"`, `"Angry"`.

Let me know if you'd like to also show the confusion matrix or F1 scores in your presentation!

Explanation:

I used ChatGPT for this section of code because I was unsure how to cut off a portion of the string to ensure that the “Emotion_” prefix wasn’t showing. I also needed this to ensure that the classification report could work properly as it was giving me errors related to the strings.

Durga Sandeep

💡 What is `GridSearchCV` in Machine Learning?

`GridSearchCV` is a method provided by `scikit-learn` that helps find the best hyperparameters for a machine learning model by exhaustively searching through a manually specified subset of the hyperparameter space. It stands for **Grid Search with Cross-Validation**.

🧠 Why use `GridSearchCV`?

Machine learning models often depend on **hyperparameters**—like `C`, `gamma`, `kernel` in SVM or `max_depth`, `n_estimators` in Random Forest—which significantly affect performance.

Manually trying different combinations is:

- Inefficient
- Error-prone
- Not guaranteed to find the best settings

`GridSearchCV` solves this with a systematic search and evaluation.



What is the Linear Kernel in Machine Learning?

The **linear kernel** is one of the simplest types of kernel functions used in algorithms like **Support Vector Machines (SVMs)**. It's used when your data is **linearly separable**—meaning it can be divided into classes by a straight line (2D), plane (3D), or hyperplane (higher dimensions).

Formula of Linear Kernel

For two vectors x and y , the linear kernel is:

$$K(x, y) = x^T y$$

This is just the **dot product** of two vectors.

What does it do?

- In **SVM**, the kernel function computes similarity  between two data points.

Comparison: Linear vs RBF Kernel

Feature	Linear Kernel	RBF Kernel
Linearity	Works with linear data	Works with nonlinear data
Speed	Faster	Slower
Flexibility	Low	High
Parameters	<code>C</code>	<code>C</code> , <code>gamma</code>
Overfitting risk	Lower	Higher (if γ is too high)

✓ Models That Can Work Better Than SVM

Model	When It Might Be Better	Key Strengths
Random Forest	- High-dimensional or noisy data - Nonlinear problems	- Handles missing values - Robust to outliers - Feature importance
Gradient Boosting (e.g., XGBoost, LightGBM)	- When accuracy is top priority - Complex relationships	- Highly accurate - Customizable loss functions - Handles overfitting well
Logistic Regression	- Linearly separable data - Simpler problems	- Interpretable - Fast and low-resource
Neural Networks	- Large datasets - Complex, non-linear data (e.g., images, speech)	- Learns intricate patterns - Scalable to deep learning
k-Nearest Neighbors (k-NN)	- Small datasets - Local pattern learning	- Simple and intuitive - No training time
Naive Bayes	- Text classificati↓ - High-dimensional sparse data	- Fast - Performs well on text/email

✓ Interpreting AUC Values

AUC Score	Interpretation
0.90 – 1.0	Excellent
0.80 – 0.90	Good
0.70 – 0.80	Fair
0.60 – 0.70	Poor
0.50 – 0.60	Fail (no discrimination)

Sample ROC Curve Code

python

 Copy  Edit

```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Assuming y_test is true labels and y_prob is predicted probabilities
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label=f"ROC curve (AUC = {roc_auc:.2f})")
plt.plot([0, 1], [0, 1], 'k--') # Random guess line
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.grid(True)
plt.show()
```